

Projekt Kosy

Dokumentacja Open API



Rok III / Semestr V

Daniel Kozak

Emilia Małecka

Kamil Pasierski

Oskar Kuklewski

Wiktor Semp

Collegium Witelona - Uczelnia Państwowa
Wydział Nauk Technicznych i Ekonomicznych

22 stycznia 2026

Spis treści

1	Uwierzytelnianie i bezpieczeństwo	4
1.1	Metody uwierzytelniania	4
1.2	Przykłady token'ów	4
2	Endpointy API	5
2.1	Reset hasła	5
2.1.1	Zażądanie resetu hasła	5
2.1.2	Ustalenie nowego hasła	6
2.2	Logowanie przez Google OAuth2	7
2.3	Użytkownicy	8
2.3.1	Lista wszystkich użytkowników	8
2.3.2	Dane aktualnego użytkownika	9
2.4	Administracja	10
2.4.1	Lista oczekujących ticket'ów	10
2.4.2	Akcja na zgłoszeniu	11
2.4.3	Powiadomienia użytkownika	12
2.4.4	Oznaczenie powiadomienia jako przeczytane	13
2.5	Ticket'y (zgłoszenia)	14
2.5.1	Utworzenie nowego zgłoszenia	14
2.6	Statystyki systemowe	15
2.6.1	Statystyki globalne	15
2.6.2	Top kluby w konfliktach	16
2.7	Relacje między klubami	17
2.7.1	Lista wszystkich relacji	17
2.7.2	Aktualizacja relacji	18
2.8	Rejestracja użytkownika	19
2.8.1	Rejestracja nowego konta	19
2.8.2	Aktywacja konta	20
2.9	Uwierzytelnianie konta	21
2.9.1	Uzyskanie tokena autoryzacyjnego	21
2.9.2	Profil użytkownika	22
2.10	Popularne kluby	23
2.10.1	Top 6 klubów	23
2.10.2	Inkrementacja punktów klubu	24
2.11	Kluby	25
2.11.1	Lista wszystkich klubów	25
2.11.2	Wyszukiwanie klubów z paginacją	26
2.11.3	Szczegóły klubu	27

2.11.4	Ulubiony klub użytkownika	28
2.11.5	Autouzupełnianie nazw klubów	29
2.12	Terytorium klubów	30
3	Przykłady użycia	31
3.1	cURL	31
3.2	Python	32

1 Uwierzytelnianie i bezpieczeństwo

1.1 Metody uwierzytelniania

1. Token JWT
2. Google OAuth2
3. Token autoryzacyjny

1.2 Przykłady token'ów

```
# JWT token:  
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...  
  
# token autoryzacyjny:  
Authorization: Token 9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b
```

2 Endpointy API

2.1 Reset hasła

2.1.1 Zażądanie resetu hasła

Po wysłaniu request'a o reset hasła, wysłane zostanie link do zresetowania hasła na podany adres email.

POST - Zażądanie resetu:

```
POST /api/password-reset/  
Content-Type: application/json  
  
{  
  "email": "user@example.com"  
}
```

Odpowiedź (200 OK):

```
{  
  "message": "Jeśli email istnieje, wysłano link."  
}
```

Proces:

1. System sprawdza czy użytkownik istnieje
2. Generuje token resetu
3. Wysyła link na adres email
4. Format link'a: `/reset-password/[uid]/[token]`

2.1.2 Ustalenie nowego hasła

Po potwierdzeniu reset'u hasła przez zweryfikowanie mail'owe, wprowadzamy nowe hasło, które wywołuje następną metodę.

POST - Zmiana hasła:

```
POST /api/password-reset/confirm/  
Content-Type: application/json  
  
{  
  "uid": "MQ",  
  "token": "abc123def456",  
  "password": "NoweHaslo123!"  
}
```

Odpowiedź (200 OK):

```
{  
  "message": "Hasło zmienione poprawnie"  
}
```

Błędy (400 Bad Request):

```
{  
  "error": "Nieprawidłowy link"  
}
```

```
{  
  "error": "Token wygasł lub jest nieprawidłowy"  
}
```

Proces:

1. Użytkownik wprowadza nowe hasło
2. System sprawdza token oraz czy hasło spełnia minimalne wymogi
3. System przypisuje nowe hasło do użytkownika przez UID lub zwraca błąd w razie problemu

2.2 Logowanie przez Google OAuth2

Ten endpoint umożliwia logowanie przez konto Google.

POST - Logowanie Google:

```
POST /api/google-login/  
Content-Type: application/json  
  
{  
  "token": "eyJhbGciOiJSUzI1NiIsImtpZCI6IjFl...google_id_token"  
}
```

Odpowiedź (200 OK):

```
{  
  "access": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",  
  "refresh": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9..."  
}
```

Błędy (400 Bad Request):

```
{  
  "error": "Brak tokenu Google"  
}
```

```
{  
  "error": "Nieprawidłowy token Google"  
}
```

Proces:

1. System pobiera token ID z Google Sign-In
2. Token jest weryfikowany przez Google
3. Tworzy nowe konto lub loguje użytkownika
4. Generowane są tokeny JWT

2.3 Użytkownicy

2.3.1 Lista wszystkich użytkowników

Endpoint zwraca listę wszystkich użytkowników.

GET - Lista użytkowników:

```
GET /api/users/  
Authorization: Bearer <token>
```

Odpowiedź (200 OK):

```
[  
  {  
    "id": 1,  
    "username": "admin",  
    "email": "admin@example.com",  
    "is_staff": true,  
    "is_active": true,  
    "date_joined": "2024-01-15T10:30:00Z"  
  },  
  {  
    "id": 2,  
    "username": "jan_kowalski",  
    "email": "jan@example.com",  
    "is_staff": false,  
    "is_active": true,  
    "date_joined": "2024-02-20T14:45:00Z"  
  }  
]
```


2.3.2 Dane aktualnego użytkownika

Endpoint pokazuje lub aktualizację dane użytkownika.

GET - Pobranie danych:

```
GET /api/users/me/  
Authorization: Bearer <token>
```

PATCH - Aktualizacja danych:

```
PATCH /api/users/me/  
Authorization: Bearer <token>  
Content-Type: application/json  
  
{  
  "email": "nowy@example.com",  
  "username": "nowa_nazwa"  
}
```

Odpowiedź (200 OK):

```
{  
  "id": 2,  
  "username": "nowa_nazwa",  
  "email": "nowy@example.com",  
  "is_staff": false,  
  "is_active": true,  
  "date_joined": "2024-02-20T14:45:00Z"  
}
```

2.4 Administracja

2.4.1 Lista oczekujących ticket'ów

Endpoint zwraca listę ticket'ów czekających na decyzję administratora.

GET - Oczekujące zgłoszenia:

```
GET /api/admin/pending/  
Authorization: Bearer <admin_token>
```

Odpowiedź (200 OK):

```
[  
  {  
    "id": 15,  
    "date": "21.01.2025, 14:30",  
    "reporter": "jan_kowalski",  
    "clubA": {  
      "name": "Legia Warszawa",  
      "logoUrl": "/media/legia.jpg"  
    },  
    "clubB": {  
      "name": "Lech Poznań",  
      "logoUrl": "/media/lech.jpg"  
    },  
    "relation": "kosa"  
  }  
]
```

Te zgłoszenia są weryfikowane przez admin'a aby aktualizować sprawdzone relacje klubów.

2.4.2 Akcja na zgłoszeniu

Endpoint akceptuje lub odrzuca ticket.

POST - Akceptacja zgłoszenia:

```
POST /api/admin/15/action/  
Authorization: Bearer <admin_token>  
Content-Type: application/json  
  
{  
  "action": "approve"  
}
```

POST - Odrzucenie zgłoszenia:

```
POST /api/admin/15/action/  
Authorization: Bearer <admin_token>  
Content-Type: application/json  
  
{  
  "action": "reject"  
}
```

Odpowiedź (200 OK) - akceptacja:

```
{  
  "message": "Zgłoszenie zatwierdzone"  
}
```

Odpowiedź (200 OK) - odrzucenie:

```
{  
  "message": "Zgłoszenie odrzucone"  
}
```

2.4.3 Powiadomienia użytkownika

Endpoint zwraca listę powiadomień dla zalogowanego użytkownika.

GET - Lista powiadomień:

```
GET /api/admin/notifications/  
Authorization: Bearer <token>
```

Odpowiedź (200 OK):

```
[  
  {  
    "id": 42,  
    "content": "Relacja między Legia Warszawa a Lech Poznań zosta  
      ↪ ła zaakceptowana przez administratora.",  
    "timestamp": "21.01.2025, 15:45"  
  },  
  {  
    "id": 41,  
    "content": "Twój klub został dodany do ulubionych.",  
    "timestamp": "20.01.2025, 09:30"  
  }  
]
```

Te odpowiedzi są wykorzystywane przez frontend aby wyświetlać powiadomienia.

2.4.4 Oznaczenie powiadomienia jako przeczytane

Endpoint umożliwia oznaczenie powiadomienia jako przeczytane.

PATCH - Oznaczenie jako przeczytane:

```
PATCH /api/admin/change_is_read/  
Authorization: Bearer <token>  
Content-Type: application/json  
  
{  
  "id": 42  
}
```

Odpowiedź (200 OK):

```
{  
  "id": 42,  
  "content": "Relacja między Legia Warszawa a Lech Poznań została  
    ➔ zaakceptowana przez administratora.",  
  "timestamp": "21.01.2025, 15:45"  
}
```

2.5 Ticket'y (zgłoszenia)

2.5.1 Utworzenie nowego zgłoszenia

Endpoint Tworzy nowy ticket prośby o aktualizację relacji między klubami.

POST - Utworzenie zgłoszenia:

```
POST /api/tickets/
Authorization: Bearer <token>
Content-Type: application/json

{
  "club_a": "Legia Warszawa",
  "club_b": "Lech Poznań",
  "description": "Kluby mają długoletni konflikt...",
  "relation": "kosa"
}
```

Odpowiedź (201 Created):

```
{
  "id": 15,
  "club_a": "Legia Warszawa",
  "club_b": "Lech Poznań",
  "description": "Kluby mają długoletni konflikt...",
  "relation": "kosa",
  "status": "pending",
  "created_at": "2025-01-21T14:30:00Z",
  "user": 42
}
```

Błędy (400 Bad Request):

```
{
  "non_field_errors": [
    "Nie można stworzyć ticketu dla tego samego klubu."
  ]
}
```

```
{
  "non_field_errors": ["Taki ticket już istnieje."]
}
```

2.6 Statystyki systemowe

2.6.1 Statystyki globalne

Endpoint zwraca globalne statystyki systemu.

GET - Statystyki globalne:

```
GET /api/stats/global/  
Authorization: Bearer <admin_token>
```

Odpowiedź (200 OK):

```
{  
  "relations": 42,  
  "clubs": 18,  
  "users": 156,  
  "tickets": 7  
}
```

2.6.2 Top kluby w konfliktach

Endpoint zwraca listę klubów z największą liczbą relacji typu "kosa"(konfliktów). Dostępny publicznie.

GET - Top konfliktów:

```
GET /api/stats/max-beefs/
```

Odpowiedź (200 OK):

```
[
  {
    "id": 5,
    "name": "Legia Warszawa",
    "path_image": "/media/legia.jpg",
    "kos_count": 8
  },
  {
    "id": 3,
    "name": "Lech Poznań",
    "path_image": "/media/lech.jpg",
    "kos_count": 6
  }
]
```


2.7 Relacje między klubami

2.7.1 Lista wszystkich relacji

Endpoint zwraca listę relacji między klubami.

GET - Lista relacji:

```
GET /api/relations/
```

Odpowiedź (200 OK):

```
[
  {
    "club_a_name": "Legia Warszawa",
    "club_b_name": "Lech Poznań",
    "relation_type": "kosa"
  },
  {
    "club_a_name": "Legia Warszawa",
    "club_b_name": "Górnik Zabrze",
    "relation_type": "zgoda"
  }
]
```

2.7.2 Aktualizacja relacji

Endpoint umożliwia administratorowi utworzenie lub aktualizację relacji między klubami.

POST - Aktualizacja relacji:

```
POST /api/relations/update/  
Authorization: Bearer <admin_token>  
Content-Type: application/json  
  
{  
  "club_a": "Legia Warszawa",  
  "club_b": "Lech Poznań",  
  "relation": "kosa"  
}
```

Odpowiedź (200 OK):

```
{  
  "status": "success",  
  "message": "Relacja utworzona jako kosa.",  
  "relation": "kosa"  
}
```

Błąd (400 Bad Request):

```
{  
  "error": "Kluby muszą być różne."  
}
```

2.8 Rejestracja użytkownika

2.8.1 Rejestracja nowego konta

Endpoint rejestruje nowego użytkownika.

POST - Rejestracja:

```
POST /api/register/  
Content-Type: application/json
```

```
{  
  "username": "nowy_uzytkownik",  
  "email": "user@example.com",  
  "password": "Haslo123!",  
  "re_password": "Haslo123!"  
}
```

Odpowiedź (201 Created):

```
{  
  "message": "Użytkownik został pomyślnie utworzony."  
}
```

Błędy (400 Bad Request):

```
{  
  "email": "Email jest już zajęty."  
}
```

```
{  
  "re_password": "Hasła nie są takie same."  
}
```

2.8.2 Aktywacja konta

Endpoint aktywuje konto użytkownika poprzez link wysłany email'em.

GET - Aktywacja:

```
GET /api/register/activate/MQ/abc123def456/
```

Przekierowanie:

- Sukces: <http://localhost:3000/?status=activated>
- Błąd: Strona błędu z komunikatem

2.9 Uwierzytelnianie konta

2.9.1 Uzyskanie tokena autoryzacyjnego

Endpoint zwraca token autoryzacji dla użytkowników zarejestrowanych przez email.

POST - Uzyskanie tokena:

```
POST /api/api-token-auth/  
Content-Type: application/json  
  
{  
  "username": "testuser",  
  "password": "Test123!"  
}
```

Odpowiedź (200 OK):

```
{  
  "token": "9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b"  
}
```

2.9.2 Profil użytkownika

Endpoint testowy zwracający odpowiedź do zalogowanego użytkownika.

GET - Informacje profilowe:

```
GET /api/profile/  
Authorization: Token 9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b
```

Odpowiedź (200 OK):

```
{  
  "message": "Hello , testuser!"  
}
```

2.10 Popularne kluby

2.10.1 Top 6 klubów

Endpoint zwraca top 6 najpopularniejszych klubów na stronie głównej.

GET - Top klubów:

```
GET /api/popular/
```

Odpowiedź (200 OK):

```
[
  {
    "id": 5,
    "name": "Legia Warszawa",
    "city": "Warszawa",
    "path_image": "/media/legia.jpg",
    "points": 1420
  },
  {
    "id": 3,
    "name": "Lech Poznań",
    "city": "Poznań",
    "path_image": "/media/lech.jpg",
    "points": 1285
  }
]
```

2.10.2 Inkrementacja punktów klubu

Endpoint zwiększa licznik punktów wyszukiwań klubu o 1. Używane w proponowaniu klubów podczas wyszukiwania.

POST - Inkrementacja punktów:

```
POST /api/popular/increment/5/
```

Odpowiedź (200 OK):

```
{
  "message": "Points incremented",
  "points": 1421
}
```


2.11 Kluby

2.11.1 Lista wszystkich klubów

Endpoint zwraca listę klubów. Używane do dropdown'ów i inicjalizacji mapy.

GET - Lista klubów:

```
GET /api/clubs/all/
```

Odpowiedź (200 OK):

```
[
  {
    "id": 1,
    "name": "Legia Warszawa",
    "city": "Warszawa",
    "desc": "Klub piłkarski...",
    "path_image": "/media/legia.jpg",
    "points": 1420
  }
]
```

2.11.2 Wyszukiwanie klubów z paginacją

Endpoint od wyszukiwania klubów.

GET - Wyszukiwanie:

```
GET /api/clubs/search/?search=war&ordering=-points&page=1
```

Odpowiedź (200 OK):

```
{
  "count": 45,
  "next": "http://localhost:8000/api/clubs/search/?page=2",
  "previous": null,
  "results": [
    {
      "id": 1,
      "name": "Legia Warszawa",
      "city": "Warszawa",
      "path_image": "/media/legia.jpg",
      "points": 1420
    }
  ]
}
```

2.11.3 Szczegóły klubu

Endpoint zwraca szczegółowe informacje o klubie.

GET - Szczegóły:

```
GET /api/clubs/5/
```

Odpowiedź (200 OK):

```
{
  "id": 5,
  "name": "Legia Warszawa",
  "city": "Warszawa",
  "desc": "Klub założony w 1916...",
  "path_image": "/media/legia.jpg",
  "points": 1420,
  "kosy": [
    {"id": 3, "name": "Lech Poznań", "path_image": "/media/lech.
      ↪ jpg"}
  ],
  "zgody": [
    {"id": 4, "name": "Górnik Zabrze", "path_image": "/media/
      ↪ gornik.jpg"}
  ],
  "neutralne": [
    {"id": 6, "name": "Śląsk Wrocław", "path_image": "/media/
      ↪ slask.jpg"}
  ]
}
```

2.11.4 Ulubiony klub użytkownika

Endpoint do ustawienia ulubionego klubu użytkownika.

GET - Pobranie ulubionego klubu:

```
GET /api/clubs/user/  
Authorization: Bearer <token>
```

POST - Ustawienie ulubionego klubu:

```
POST /api/clubs/user/  
Authorization: Bearer <token>  
Content-Type: application/json  
  
{  
  "club": 5  
}
```

Odpowiedź (200 OK):

```
{  
  "message": "Klub zaktualizowany"  
}
```

2.11.5 Autouzupełnianie nazw klubów

Endpoint proponuje kluby podczas wyszukiwania po fragmencie nazwy.

GET - Autocomplete:

```
GET /api/clubs/?q=leg
```

Odpowiedź (200 OK):

```
[
  {"id": 1, "name": "Legia Warszawa"},
  {"id": 2, "name": "Legia Soccer Schools"}
]
```

2.12 Terytorium klubów

Endpoint zwraca koordynaty terytorium klubów.

GET - Lista obszarów:

```
GET /api/area/
```

Odpowiedź (200 OK):

```
[
  {
    "id": 1,
    "polygon": [
      {"lat": 51.1079, "lng": 17.0385},
      {"lat": 51.1085, "lng": 17.0392}
    ],
    "owner_name": "Klub Sportowy Legia"
  }
]
```

3 Przykłady użycia

3.1 cURL

- Rejestracja użytkownika

```
curl -X POST http://localhost:8000/api/register/ \
  -H "Content-Type: application/json" \
  -d '{"username":"nowy","email":"user@example.com",
      "password":"Haslo123!","re_password":"Haslo123!"}'
```

- Logowanie

```
curl -X POST http://localhost:8000/api/api-token-auth/ \
  -H "Content-Type: application/json" \
  -d '{"username":"testuser","password":"Test123!"}'
```

- Pobranie klubów

```
curl -X GET "http://localhost:8000/api/clubs/all/"
```

3.2 Python

- Rejestracja

```
import requests

register_data = {
    'username': 'nowy_uzytkownik',
    'email': 'user@example.com',
    'password': 'Haslo123!',
    're_password': 'Haslo123!'
}

response = requests.post(
    'http://localhost:8000/api/register/',
    json=register_data
)
print(response.json())
```

- Logowanie

```
import requests

auth_response = requests.post(
    'http://localhost:8000/api/api-token-auth/',
    json={
        'username': 'testuser',
        'password': 'Test123!'
    }
)

token = auth_response.json()['token']
print(f"Token: {token}")
```

- Pobranie danych z autoryzacją

```
import requests

headers = {'Authorization': f'Token {token}'}
profile = requests.get(
    'http://localhost:8000/api/profile/',
    headers=headers
).json()

print(f"Wiadomość: {profile['message']}")
```